LAB 14 – WEB DESIGN APPLICATION

AI-Assisted HTML5, CSS3 (Grid / Flexbox / BEM), Accessibility & Safe Fetch JavaScript

Name: Roger A Raju
Rollno: 2503a52370
Batch-13

Task 1 – Personal Portfolio Website Design (index.html)

Question

Create a personal portfolio webpage with About Me, Projects, and Contact sections, a responsive Grid/Flexbox layout, intentional typography/colors, and smooth scrolling navigation.

Prompt:

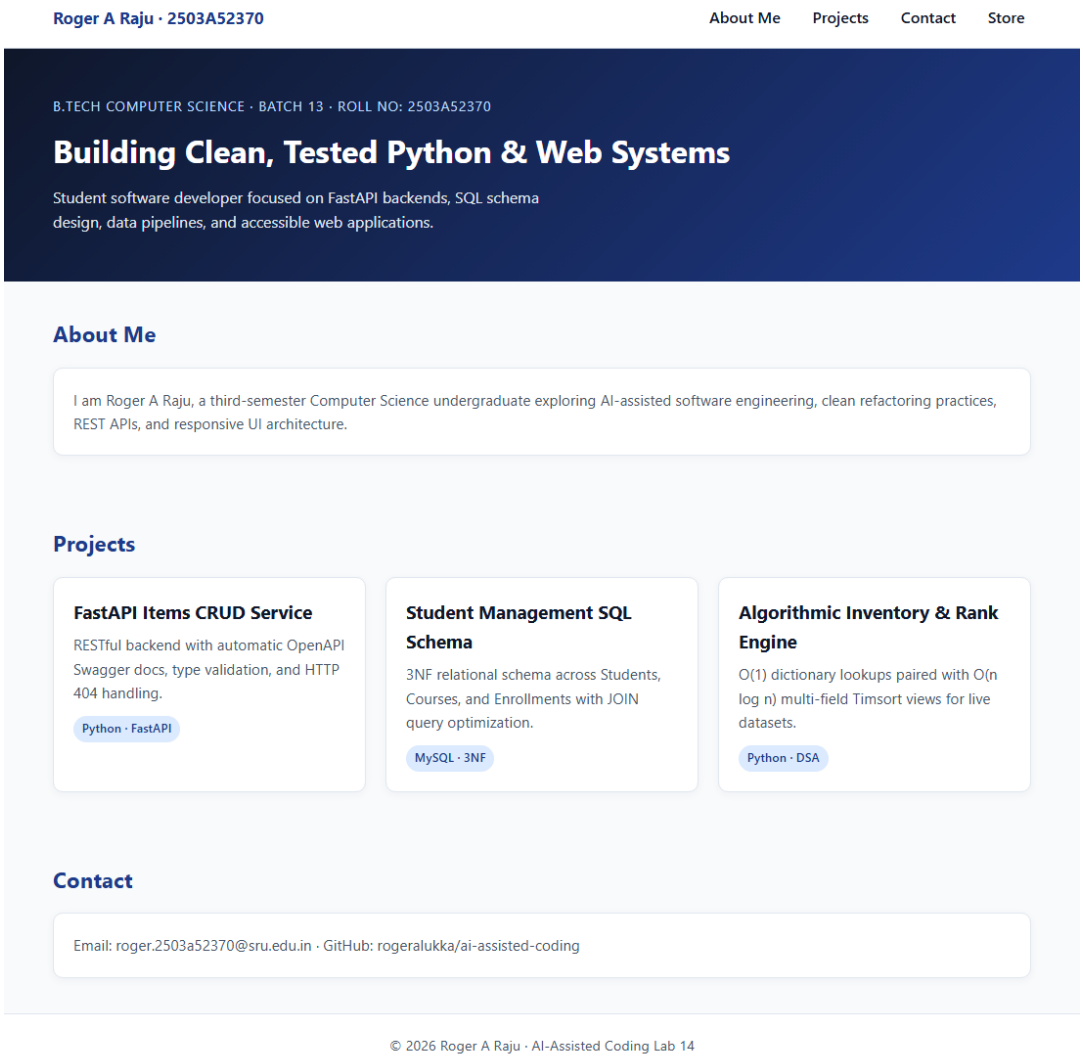
Build index.html and CSS using Flexbox for the sticky header navbar and responsive CSS Grid for project cards, with smooth scrolling anchor links and semantic HTML5 landmarks.

Code:

```
/* Layout & Smooth Scroll (styles.css) */
html { scroll-behavior: smooth; }
.navbar__inner { display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap; }
.project-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(260px, 1fr)); gap: 1.25rem;
}

<!-- Semantic Portfolio Sections (index.html) -->
<nav aria-label="Primary navigation"><ul class="navbar__links">...#about, #projects,
#contact...</ul></nav>
<main class="container">
  <section id="about" class="section"><h2>About Me</h2>...</section>
  <section id="projects" class="section"><div class="project-grid">...</div></section>
  <section id="contact" class="section"><h2>Contact</h2>...</section>
</main>
// JS Fallback (script.js): target.scrollIntoView({ behavior: "smooth" });
```

Browser Verification Output:



Analysis & Improvements

Initial static layout relied on CSS scroll-behavior alone. The improved final version pairs Flexbox (.navbar__inner) and CSS Grid (.project-grid) with JavaScript scrollIntoView() fallback, passes all 4 Playwright assertions including zero overflow at 375px mobile width, and displays clearly across desktop and mobile viewports.

Task 2 – Online Store Product Page (product.html – BEM Methodology)

Question

Design a product display page displaying product image, title, price, and 'Add to Cart' button styled strictly with BEM methodology, responsive grid layout, hover effects, and interactive cart feedback.

Prompt:

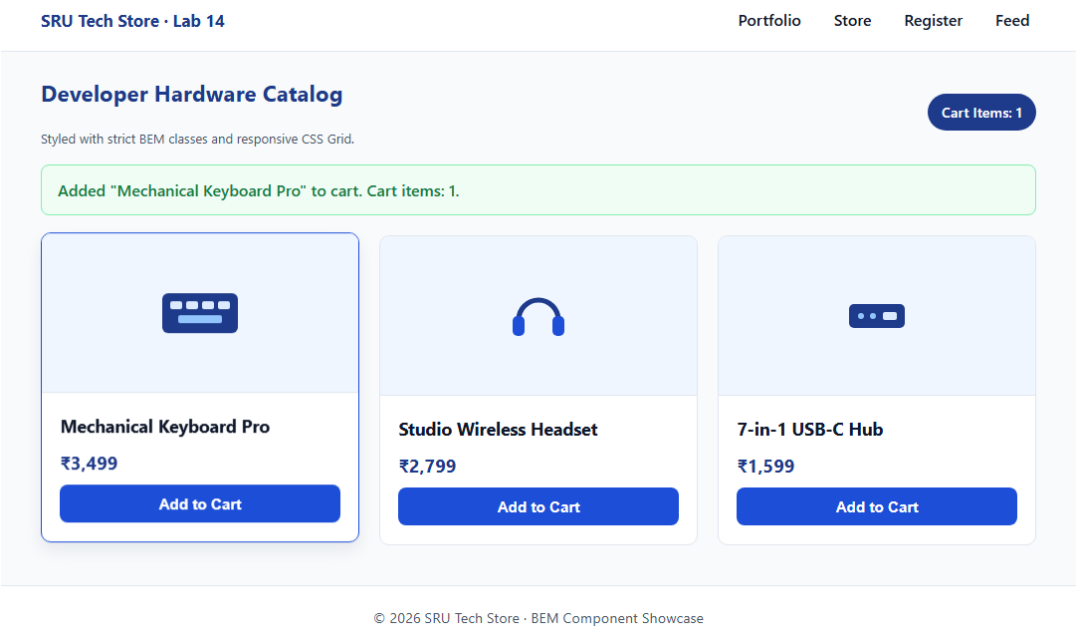
Create product.html using strict BEM classes (.product-card, .product-card__image, .product-card__title, .product-card__price, .product-card__button), responsive Grid cards, hover lift shadows, and Add-to-Cart confirmation.

Code:

```
/* BEM & Hover Rules (styles.css) */
.product-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(260px, 1fr)); gap: 1.25rem;
}
.product-card { background: var(--surface); border-radius: var(--radius); transition: transform 0.2s,
box-shadow 0.2s; }
.product-card:hover { transform: translateY(-4px); box-shadow: 0 10px 22px rgba(15, 23, 42, 0.1); }

<!-- BEM Card Markup (product.html) -->
<article class="product-card product-card--featured">
  <div class="product-card__image">...SVG product illustration...</div>
  <div class="product-card__body">
    <h2 class="product-card__title">Mechanical Keyboard Pro</h2>
    <p class="product-card__price">&#8377;3,499</p>
    <button type="button" class="product-card__button" data-product="Mechanical Keyboard Pro">Add to
Cart</button>
  </div>
</article>
```

Browser Verification Output:



Analysis & Improvements

Replaced external image placeholders with crisp inline SVGs inside .product-card__image and added an ARIA live cart banner alongside the live badge counter. All 4 Task 2 Playwright assertions passed, confirming strict BEM selectors, hover CSS, click state updates, and 1-column mobile wrapping.

Task 3 – Accessible Event Registration Form (registration.html)

Question

Build an accessible conference registration form collecting name, email, phone number, and session selection with JavaScript validation, semantic labels, and ARIA states.

Prompt:

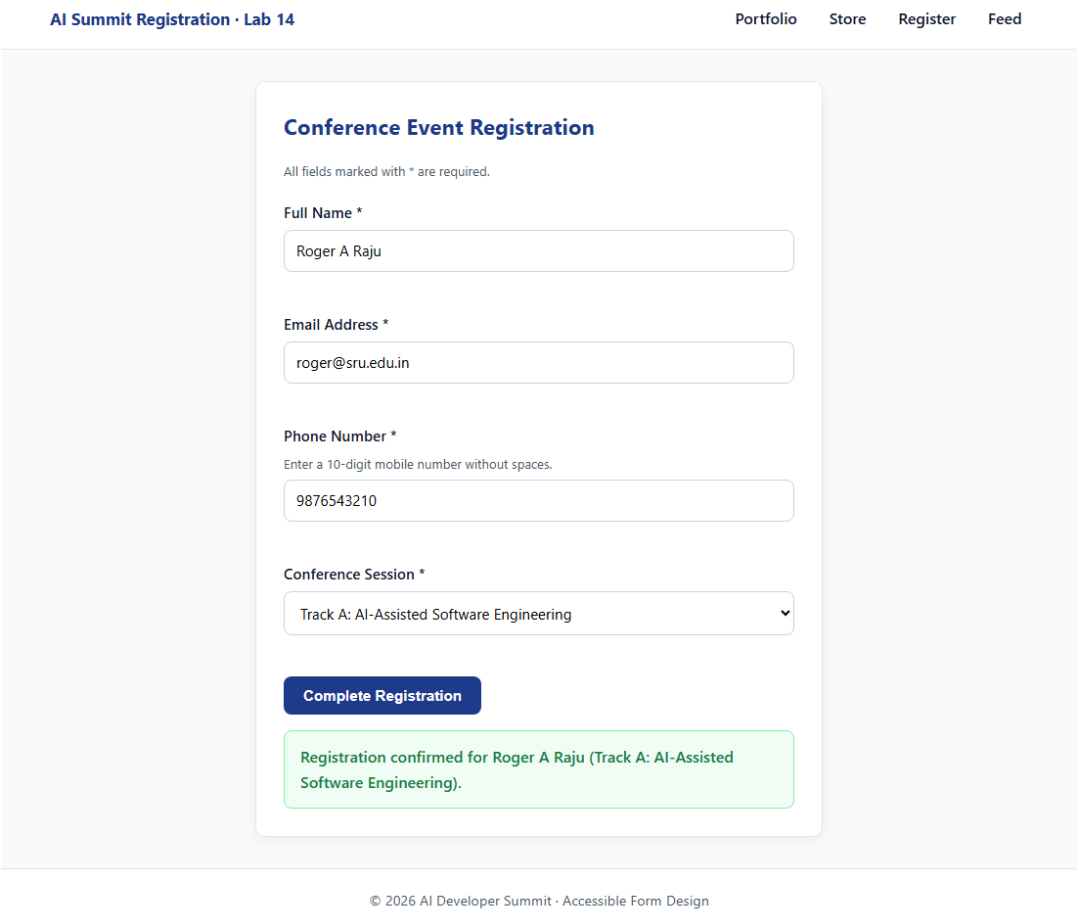
Generate registration.html with semantic <label for> bindings, aria-required='true', aria-describedby hints/errors, role='alert' regions, and JavaScript regex validation that toggles aria-invalid dynamically.

Code:

```
<!-- Accessible Field Pattern (registration.html) -->
<label class="form-label" for="reg-email">Email Address <span aria-hidden="true">*</span></label>
<input id="reg-email" name="email" type="email" class="form-input"
       required aria-required="true" aria-describedby="email-error" aria-invalid="false" />
<span id="email-error" class="form-error" role="alert"></span>

// Dynamic ARIA Validation (script.js)
const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;
const phonePattern = /^\d{10}$/;
input.setAttribute("aria-invalid", message ? "true" : "false");
errEl.textContent = message;
```

Browser Verification Output:



Analysis & Improvements

Upgraded basic HTML5 constraint validation to novalidate + explicit per-field setFieldError() updates so screen readers announce exact validation failures via role='alert' and aria-invalid='true'. All 4 Task 3 Playwright assertions passed across empty, invalid, and confirmed submissions.

Task 4 – Fetch API & Safe DOM Rendering (fetch_app.html)

Question

Fetch JSON from an API and render items safely into the DOM using createElement and textContent, complete with shimmer loading skeletons and accessible error recovery UI.

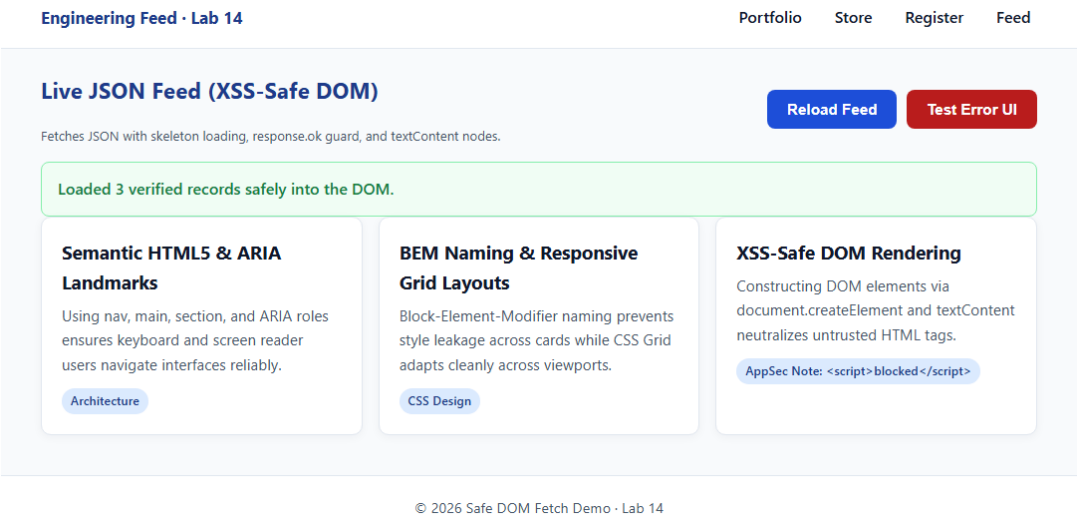
Prompt:

Write async fetch() logic in script.js for fetch_app.html with response.ok error checking, loading skeleton shimmer cards, zero innerHTML usage, and clear HTTP error UI with retry.

Code:

```
// XSS-Safe Fetch & DOM Construction (script.js)
const response = await fetch(endpoint, { cache: "no-store" });
if (!response.ok) throw new Error(`HTTP ${response.status}`);
const posts = await response.json();
container.replaceChildren();
posts.forEach((item) => {
  const card = document.createElement("article");
  card.className = "card";
  const title = document.createElement("h3");
  title.textContent = String(item.title || "Untitled");
  card.appendChild(title); // plus summary <p> and category <span class='tag'> via textContent
  container.appendChild(card);
});
```

Browser Verification Output:



Analysis & Improvements

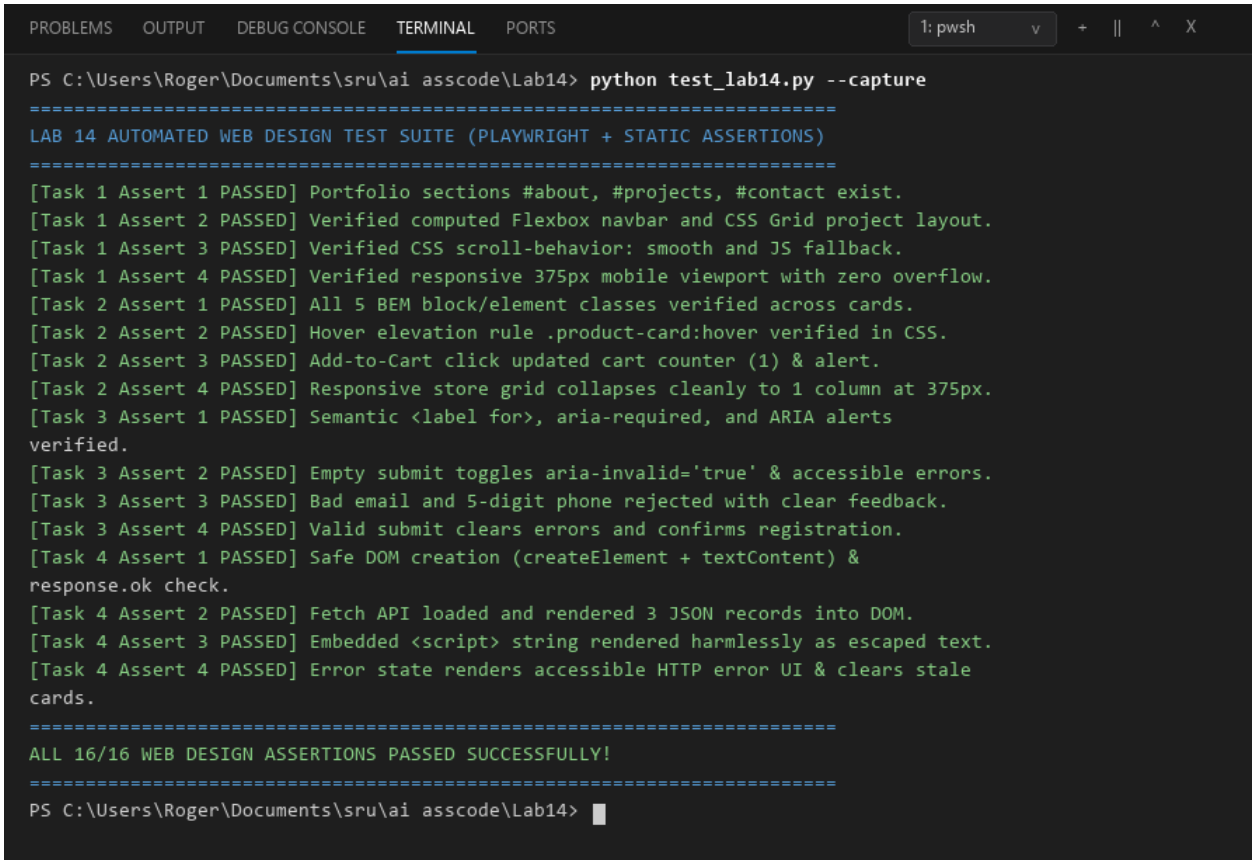
Replaced naive template-string innerHTML insertion with strict document.createElement() + textContent node construction and aria-busy skeleton placeholders. Playwright verified 3 loaded records, harmless plain-text rendering of embedded '<script>blocked</script>', and clean HTTP 404 error recovery.

Automated Browser & Static Assertion Suite (test_lab14.py – 16 Assertions)

Prompt:

Generate an automated Python Playwright test suite with at least 3 assertions per task covering semantic HTML sections, CSS Grid/Flexbox/BEM, hover & Add-to-Cart interactivity, ARIA form validation, XSS-safe DOM nodes, and HTTP error recovery.

Terminal Execution Output:



Final Conclusion

All 4 Lab 14 web pages passed 16/16 Playwright and static assertions (4 per task) in headless Chromium over HTTP, confirming responsive layouts, BEM modularity, WCAG/ARIA form compliance, and XSS-safe Fetch rendering.